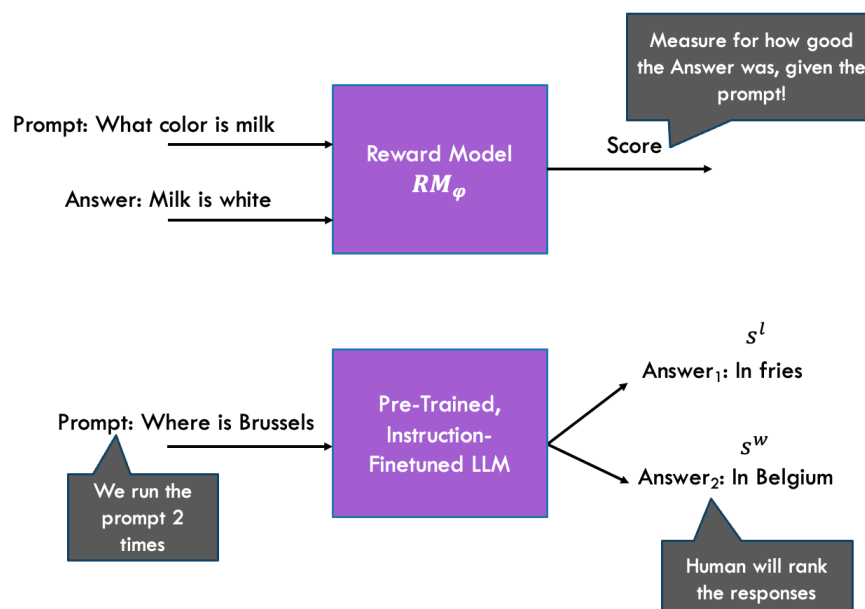


Reading 6.8: How RLHF Works

Welcome to this lesson on Reinforcement Learning from Human Feedback (RLHF)—a groundbreaking technique that shapes AI systems, such as large language models (LLMs), to align closely with human values. By the end of this video, you'll gain insights into:

1. How RLHF works.
2. Key challenges and advancements in training AI to reflect human preferences.

We could train a model that is able to score outputs, just like humans would do. The model would output a score (number) associated with an answer to a specific question (prompt). The issue however is that human feedback scores are very subjective, hence very noisy!



Instead of scoring answers directly, a human annotator is given the simple task to rank two answers given a prompt. Once the reward model RM_ϕ is trained, we can start finetuning the LLM for human feedback. The training of the reward model needs to happen first, hence we clone the pre-trained, instruction-finetuned LLM.

In RLHF, we rely on the Bradley Terry model and its associated loss function to convert a (prompt, answer) pair into a numeric reward. The Bradley-Terry loss RM_ϕ is used in pairwise comparison models to predict the probability of one item (s^w) being preferred over another (s^l). s^w and s^l represent the winning score and the losing score.

Relying on the Maximum Likelihood method we can estimate the parameters φ of the reward model such that the probability of ranking correctly the good and the bad answers is maximized.

$$Loss_{RM}(\varphi) = -E_{(x,s^w,s^l) \sim D} [\log \sigma(RM_{\varphi}(s^w) - RM_{\varphi}(s^l))]$$

We have two scenarios:

- $RM_{\varphi}(s^w) > RM_{\varphi}(s^l)$: the sigmoid (σ) will return a value >0.5 and the loss will be a small negative number
- $RM_{\varphi}(s^w) < RM_{\varphi}(s^l)$: the sigmoid (σ) will return a value <0.5 and the loss will be a very large negative number

In reality the model RM_{φ} may engage in reward hacking. Reward hacking happens when a LLM finds loopholes in the reward function to achieve high rewards without doing what's actually intended. An example would be that the Agent becomes overly polite and outputs text that makes little sense. To counter reward hacking we foresee a penalty term in the loss function of the reward model.

Makes sure that original LLM and the to be trained LLM remain close
 $\rightarrow D_{KL}$

$$R(s) = RM_{\varphi} - \beta \log \frac{\pi_{\theta}(s)}{\pi(s)} \quad \longrightarrow \quad R(s) = RM_{\varphi} - \beta D_{KL} \frac{\pi_{\theta}(s)}{\pi(s)}$$

Once the LLM is instruction-finetuned and the reward model RM_{φ} are trained, we unfreeze the LLM parameters so that the LLM can be RLHF-finetuned. The 'to be RLHF-finetuned' LLM is represented by LLM_{θ} . The overall objective of RLHF can be summarized as follows:

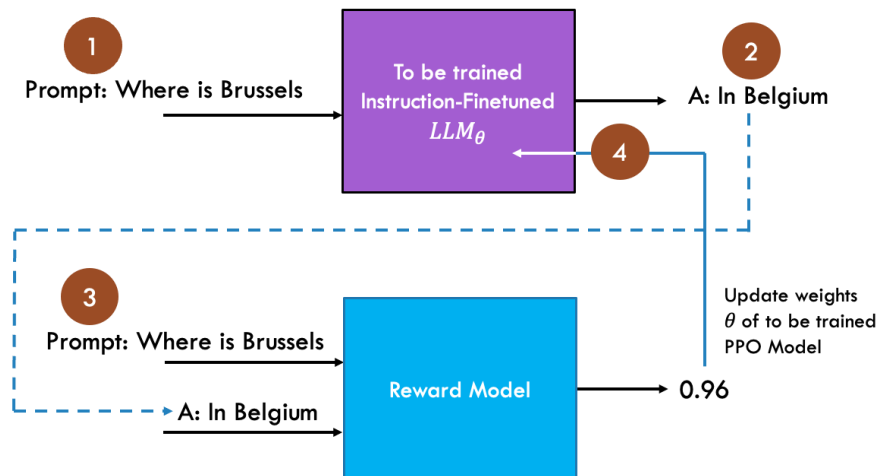
$$RLHF \text{ Objective} = \max_{\pi_{\theta}} E_{prompt(x) \sim D, y \sim \pi_{\theta}(y|x)} \left[RM_{\varphi}(x, y) - \beta D_{KL} \frac{\pi_{\theta}(s)}{\pi(s)} \right] \quad (1)$$

x: prompt/context
y: response

We take an expectation (E) as we sample over all datapoints in the dataset. We want to find the policy π parametrized by θ that maximizes the total cumulative reward. The penalty term βD_{KL} punishes text structures that deviate too much from the initial pre-trained and instruction-finetuned LLM.

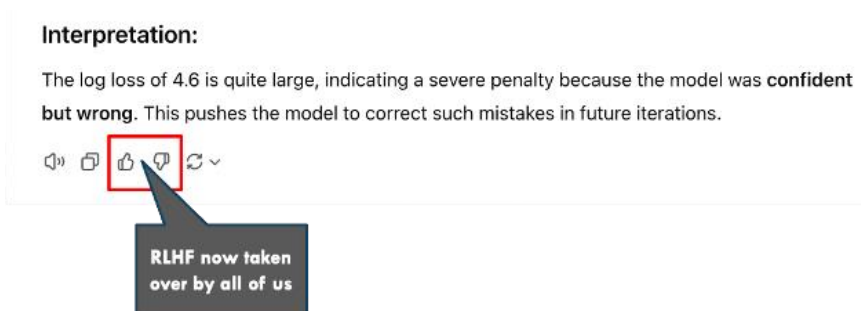
Finetuning with RLHF:

After training the reward model, we can train the LLM for human behaviour as follows:



For RLHF training, a prompt (Where is Brussels) is given to the 'to be trained' LLM_{θ} . The LLM_{θ} provides an answer (In Belgium). The answer is fed into the trained reward model that scores the answer (0.96). The score is fed back to the LLM_{θ} that will adjust its parameters. When the LLM vendor decides that the model is safe and reflecting human values, it can commercialize the model. It has been proven that a large enough reward model on enough data approaches (single) human performance.

Once the LLM is commercialized, the RLHF process continues through user feedback. When using ChatGPT, you will often find a thumbs up and/or down options to provide continuous feedback to the model and improve outcomes that are more aligned to human values.



Try it Yourself

Now that you understand the purpose of the thumbs up and down icons after ChatGPT responds to your prompt, make it a habit to use them and help improve ChatGPT's performance.